2
Using PCA reduces the dimensions of linearly inseparable data.

```
from sklearn.datasets import make_classification
from sklearn.decomposition import PCA

X, y = make_classification(n_samples=500, n_features=4,
n_informative=2,
                           n_redundant=0, n_clusters_per_class=1,
random_state=42)

print("Original Data (First 5 samples):")
print(X[:5])
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)


print("\nPCA Reduced Data (First 5 samples):")
print(X_pca[:5])

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='coolwarm', s=10)
plt.title("PCA on make_classification Data")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.grid(True)
plt.show()
```

3
Write a program for reducing the dimensionality of sparse feature matrices.

```
from sklearn.decomposition import TruncatedSVD
import numpy as np
from scipy.sparse import csr_matrix

X = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

sparse_matrix = csr_matrix(X)

print("Original sparse matrix:")
print(sparse_matrix)
```

```
n_components = 2

tsvd = TruncatedSVD(n_components=n_components)

reduced_matrix = tsvd.fit_transform(sparse_matrix)

print("\nReduced dimensionality matrix:")
print(reduced_matrix)
```

4
Perform Linear Regression.

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

model = LinearRegression()
model.fit(X, y)

y_pred = model.predict(X)

print("Intercept (c):", model.intercept_)
print("Coefficient (m):", model.coef_)
print("Predicted values:", y_pred)

plt.scatter(X, y, color='blue', label='Actual Data (y)')
plt.plot(X, y_pred, color='red', label='Regression Line')
plt.xlabel("Independent Variable (X)")
plt.ylabel("Dependent Variable (y)")
plt.title("Linear Regression: X vs y")
plt.legend()
plt.grid(True)
plt.show()
```

5
Perform Logistic Regression.

```
# prompt: Perform Logistic Regression.
```

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

x = np.arange(10).reshape(-1, 1)
y = np.array([0, 0, 0, 0, 1, 1, 1, 1, 1, 1])

model = LogisticRegression(solver='liblinear', random_state=0)
model.fit(x, y)

model.predict(x)

model.score(x, y)
```

6

Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics

try:
    df = pd.read_csv('data.csv')
except FileNotFoundError:
    print("Error: 'data.csv' not found. Please upload your data file.")
    data = {'feature1': [1, 2, 3, 4, 5],
            'feature2': [6, 7, 8, 9, 10],
            'target': [0, 0, 1, 1, 0]}
    df = pd.DataFrame(data)
    print("Using a dummy dataset for demonstration.")


X = df.drop(df.columns[-1], axis=1)
y = df[df.columns[-1]]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

```
model = GaussianNB()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = metrics.accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
```

7

Write a program to implement SVM algorithm to classify the iris dataset.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

iris = datasets.load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

svm_classifier = SVC(kernel='linear', random_state=42)
svm_classifier.fit(X_train, y_train)

y_pred = svm_classifier.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)
```

```
print(f"Accuracy: {accuracy}")
print(f"Confusion Matrix:\n{conf_matrix}")
print(f"Classification Report:\n{class_report}")
```

8

Write a program to implement Random Forest to classify the iris data set. Print both correct and wrong predictions.

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

correct_predictions = []
wrong_predictions = []

for i in range(len(y_test)):
  if y_pred[i] == y_test[i]:
    correct_predictions.append((X_test[i], y_test[i], y_pred[i]))
  else:
    wrong_predictions.append((X_test[i], y_test[i], y_pred[i]))

print("Correct Predictions:")
for prediction in correct_predictions:
  print(f"Features: {prediction[0]}, True Label:
{iris.target_names[prediction[1]]}, Predicted Label:
{iris.target_names[prediction[2]]}")

print("\nWrong Predictions:")
for prediction in wrong_predictions:
```

```
  print(f"Features: {prediction[0]}, True Label:
{iris.target_names[prediction[1]]}, Predicted Label:
{iris.target_names[prediction[2]]}")
```

9

Implement K means Clustering algorithm on dataset. Visualise the results for the same.

```
# prompt: Implement K means Clustering
# algorithm on dataset. Visualise
# the results for the same.

from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import numpy as np

data = np.random.rand(100, 2)

k = 3

kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
kmeans.fit(data)

labels = kmeans.labels_

centers = kmeans.cluster_centers_

plt.figure(figsize=(8, 6))

plt.scatter(data[:, 0], data[:, 1], c=labels, cmap='viridis', s=50,
alpha=0.7)

plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=1,
marker='X', label='Cluster Centers')

plt.title('K-Means Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True)
plt.show()
```

10
Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.